1 MINLP Modeling

Table 1.1: Constants and Variables for One Level

Constants	
T_{SL}	time to lookup a k-mer in vector S
T_H	time to lookup a k-mer in the hash table
$P_K(m,t)$	empirically estimated value for a k-mer with a minimizer of length m to have a frequency less than t
Variables	
$\mathbf{p_K} \in [0,1]$	probability of a k-mer to have a minimizer with frequency $\leq t$
$\mu_m \in \{0,1\}$	binary selector for every minimizer length $m \in \mathcal{M}$
$\tau_t \in \{0,1\}$	binary selector for every threshold $t \in \mathcal{T}$
$\mathbf{m}$	optimal minimizer length
$\mathbf{t}$	optimal threshold

One Level

As a first approach we model the runtime and space consumption for only one level.
Let $\mathcal{M}$ be the Set of all minimizer length and $\mathcal{T}$ the set of all thresholds to be considered.

Runtime

To lookup a k-mer q with minimizer $m(q)$ of frequency $f(m(q))$ on one level, first the bit vector R is checked in order to determin whether $1 \leq f(m) \leq threshold$. This can be ignored when modeling the lookup runtime as this is practically not influenced by the parameters. reason
For $\mathbf{p_K}$ of the k-mers, the corresponding bit in R will be set, which results in a lookup in S. If $m(q)$ does not appear at all or has a $f(m(q)) > t$, a lookup in the hash table will follow. The lookup time T_{lookup} for one level can thus be modeled as:

$$T_{lookup} = T_{SL} * \mathbf{p_K} + T_H * (1 - \mathbf{p_K})$$

In order to determine m and t we introduce binary selector variables μ_m, τ_t for each possible $m \in \mathcal{M}$ and $t \in \mathcal{T}$.
μ_m should be 1 exactly when the optimal minimizer length is m.
τ_t should be 1 exactly when the optimal threshold is t.
The selectors for m and t must be binary and mutually exclusive:

$$\mu_m \in 0,1 \mid \forall m \in \mathcal{M}$$
$$\tau_t \in 0,1 \mid \forall t \in \mathcal{T}$$
$$\sum_{m \in \mathcal{M}} \mu_m = 1$$
$$\sum_{t \in \mathcal{T}} \tau_t = 1$$

For a fixed minimizer length m and threshold t let $P_K(m,t)$ be the empirically determined percentage of k-mers q that have a minimizer with $f(m(q)) < t$.
We can then model $\mathbf{p_K}$ with:

$$\mathbf{p_K} = \sum_{m \in \mathcal{M}} \left(\mu_m \sum_{t \in \mathcal{T}} P_K(m,t) * \tau_t \right)$$

$\mathbf{m}$ and $\mathbf{t}$ can then be determined by:

$$\mathbf{m} = \sum_{m \in \mathcal{M}} \mu_m * m$$
$$\mathbf{t} = \sum_{t \in \mathcal{T}} \tau_t * t$$

The above results in the following MINLP (Mixed Integer Non Linear Program):

MINIMIZE

$$T_{lookup} = {T_{SL}}^1 * \mathbf{p_K} + T_H * (1 - \mathbf{p_K})$$

SUBJECT TO

$$\mathbf{p_K} = \sum_{m \in \mathcal{M}} \left(\mu_m \sum_{t \in \mathcal{T}} P_K(m,t) * \tau_t \right)$$
$$\mu_m \in 0, 1 \mid \forall m \in M$$
$$\tau_t \in 0, 1 \mid \forall t \in T$$
$$\sum_{m \in M} \mu_m = 1$$
$$\sum_{t \in T} \tau_t = 1$$
$$\mathbf{m} = \sum_{m \in \mathcal{M}} \mu_m * m$$
$$\mathbf{t} = \sum_{t \in \mathcal{T}} \tau_t * t$$

For now we will assume that $T_{SL} \approx 40ns$ and $T_H \approx 20ns$.
Solving this MINLP will result in a Solution where the μ_m, τ_t corresponding to the smallest $m \in \mathcal{M}$, $t \in \mathcal{T}$ will be set to one. This is as expected, since this leads to the smallest possible $\mathbf{p_K}$. This minimizes the k-mers found in the vector S and thus maximizes the lookups in the (faster) hash table.

However this also results in a high space consumption, because each entry in the hash table containing the minimizer as a key and the offset as its value is significantly larger than an entry in S which only contains offsets.

In order to take that into account, we have to add a constraint modeling the space consumption.

Space Consumption

The space consumption of the Data Structure used by RSHash can be split into that of the Vectors R, O and S as well as the hash table H.

Bit Vectors R and S

R and S can likely be disregarded when modeling the space consumption. explain why
R is a Bit Vector of length $4^{\mathbf{m}}$ that contains n 1s marking the existence of each encountered minimizer. Using Elias-Fano-Encoding, its size can be compressed to possibly model n?

$$n\lceil log(4^{\mathbf{m}}/n) \rceil + 3n$$

Offsetvector O

O contains one entry of size $log|T|$ for the offset of every minimizer with frequency $\leq t$ in T.
Every minimizer of length $\mathbf{m}$ covers an average of $\frac{k-\mathbf{m}+2}{2}$ consecutive k-mers. We can thus model the bits per k-mer in O as:

$$bpk_O = \mathbf{p_K} * \frac{2}{k - \mathbf{m} + 2} * log|T|$$

Hash table H

For $1 - \mathbf{p_K}$ of the k-mers q in T H contains an entry $(key : q, value : offset(q))$
Each entry has size $size(key) + size(value) = 64 + log|T|$.

Table 1.2: Constants and Variables for Several Level

Constants	
T_{SL}	time to lookup a k-mer in vector S
T_H	time to lookup a k-mer in the hash table
$P_K(m,t)$	empirically estimated value for a k-mer with a minimizer of length m to have a frequency less than t
Variables	
$\mathbf{p_K^l} \in [0,1]$	probability of a k-mer to have a minimizer with frequency $\leq t$ on level l
$\mu_m^l \in \{0,1\}$	binary selector for every minimizer length $m \in \mathcal{M}$ on level l
$\tau_t^l \in \{0,1\}$	binary selector for every threshold $t \in \mathcal{T}$ on level l
$\mathbf{m}^l$	optimal minimizer length on level l
$\mathbf{t}^l$	optimal threshold on level l

The bits per k-mer required for the hash table can thus be modeled by

$$bpk_H = (1 - \mathbf{p_K}) * (64 + log|T|)$$

Several Levels

Let

Runtime

RSHash uses several levels, to make use of the skewniss of minimizer distribution.
Each level l has a pair of parameters $\mathbf{m}^l, \mathbf{t}^l$ which result in a probability $\mathbf{p_K^l}$ of a k-mer belonging to level l.
For now, we still assume, that the time for a lookup in the vectors R, S as well as the hash table H are (practically) independent of the parameters $\mathbf{m}^l, \mathbf{t}^l$. reason

When using L levels, the probability of a k-mer belonging to a higher level is dependent on the probability of k-mers belonging to the levels below.
On each level $1 \leq l \leq L$, for $\mathbf{p_K^l}$ of the k-mers, the lookup will be completed by finding a value in the offset vector on that level whilst the remaining $(1 - \mathbf{p_K^l})$ queries result in a lookup on the next level.
We can model the lookup time for all levels $1 \leq l < L$ levels recursively as:

$$T_{lookup}^l = \begin{cases} \mathbf{p_K^l} * T_{SL} + (1 - \mathbf{p_K^l}) * T_{lookup}^{l+1} & | \, 1 \leq l \leq L \\ T_H & | \, l = L + 1 \end{cases}$$

For $L = 3$ levels, this results in:

$$\begin{aligned} T_{lookup} = \; & \mathbf{p_K^1} * T_{SL} \\ & + (1 - \mathbf{p_K^1}) * \Big(\mathbf{p_K^2} * T_{SL} \\ & \qquad + (1 - \mathbf{p_K^2}) * \Big(+ \mathbf{p_K^3} * T_{SL} \\ & \qquad\qquad + (1 - \mathbf{p_K^3}) * \end{aligned}$$

Because $\mathbf{m}^l$ and $\mathbf{t}^l$ can be different on every level l, a different set of binary selector variables μ_m^l and τ^l is needed.
The constraints for each level are the same as for one level:

$$\begin{aligned} & \mu_m^l \in 0,1 && | \, \forall (l,m) \in (\mathcal{L} \times \mathcal{M}) \\ & \tau_t^l \in 0,1 && | \, \forall (l,t) \in (\mathcal{L} \times \mathbf{t}) \\ & \textstyle\sum_{m \in \mathcal{M}} \mu_m^l = 1 && | \, \forall l \\ & \textstyle\sum_{t \in \mathcal{T}} \tau_t^l = 1 && | \, \forall l \end{aligned}$$

For every level l $\mathbf{p_K^l}, \mathbf{m}^l$ and $\mathbf{t}^l$ can also be modeled analogously to having only one level: this only holds under the assumption, that the mini-

$$\mathbf{p}_\mathbf{K}^l = \sum_{m \in \mathcal{M}} \left(\mu_m^l \sum_{t \in \mathcal{T}} P_K(m,t) * \tau_t^l \right) \quad | \; \forall l \in \mathcal{L}$$

$$\mathbf{m}^l = \sum_{m \in \mathcal{M}} \mu_m^l * m \quad | \; \forall l \in \mathcal{L}$$

$$\mathbf{t}^l = \sum_{t \in \mathcal{T}} \tau_t^l * t \quad | \; \forall l \in \mathcal{L}$$

paragraph/section about the complexity of finding the solution

For these reasons, an approach yielding a solution that is a certain approximation of the optimum but takes less time to find it could be of interest.
An easy adaption to speed up the search for empirical values would be to only sample part of the genome. By sampling sequences of length at a large number of random locations, the probability of choosing a bad representation becomes highly unlikely.
By sampling only part of the genome, we can speed up the finding of empirical values by a factor of / or even "get it down to a constant" (that is for a fixed number of sample locations - regardless of the genomes size. this however results in a growing error for larger genomes).

what length

how lorge

factor

Space Consumption

Offsetvector O

should I introduce an index for the minimizer function as it differs for every level? e.g. $m^l(q)$

For several levels, the offset vector for level l only contains an entry for a k-mer q, if the k-mers minimizer $m(q)$ has a frequency $f(m(q)) > t^i$ for all levels $1 \leq i < l$ and $f(m(q)) \leq t^l$.
The bits per k-mer for the Offset vector on level l can thus be modeled as:

$$bpk_{O^l} = \prod_{i=1}^{l-1} (1 - \mathbf{p}_\mathbf{K}^i) * \mathbf{p}_\mathbf{K}^l * \frac{2}{k - \mathbf{m}^l + 2} * log|T|$$

Hash table H

The hash table only contains an entry k-mers q, for which $f(m(q)) > t^l$ for all levels $1 \leq l \leq L$.
The bits per k-mer for the hash table can thus be modeled as

$$bpk_H = \prod_{l=1}^{L} (1 - \mathbf{p}_\mathbf{K}^l) * (64 + log|T|)$$

Limitations

over what

does it?

how much, why?

This approach of solving the constructed model has the advantage () of finding optimal results .
However for large genomes, finding the empirical values $P_K(m,t)$ becomes increasingly time consuming and needs a lot of RAM .